



UNIVERSIDADE FEDERAL DO PIAUÍ  
CAMPUS SENADOR HELVÍDIO NUNES DE BARROS  
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

**SIMULAÇÃO DE CONCORRÊNCIA COM  
THREADS:  
IMPLEMENTAÇÃO DO PROBLEMA DOS  
FILÓSOFOS JANTANDO**

Allana Camily de Sousa Martins

Janiele de Jesus Barbosa

José Henrique dos Santos Reis

Lucas Gonçalves de Jesus Pereira

Professor: Dennis Sávio

Picos – PI

2026



# **SIMULAÇÃO DE CONCORRÊNCIA COM THREADS:**

## **IMPLEMENTAÇÃO DO PROBLEMA DOS FILÓSOFOS JANTANDO**

Relatório apresentado como requisito parcial para obtenção de nota na disciplina de Sistemas Operacionais do curso de Bacharelado em Sistemas de Informação da Universidade Federal do Piauí.

Professor: Dennis Sávio

Picos – PI

2026



# Sumário

|          |                                           |          |
|----------|-------------------------------------------|----------|
| <b>1</b> | <b>Introdução</b>                         | <b>2</b> |
| <b>2</b> | <b>Ideias Criativas</b>                   | <b>2</b> |
| <b>3</b> | <b>Organização do Projeto</b>             | <b>3</b> |
| <b>4</b> | <b>Metodologia</b>                        | <b>3</b> |
| <b>5</b> | <b>Implementação das Funções</b>          | <b>4</b> |
| 5.1      | Função run . . . . .                      | 4        |
| 5.2      | Função main . . . . .                     | 4        |
| 5.3      | Uso de Semáforos . . . . .                | 5        |
| 5.4      | Uso de Mutex (Locks) . . . . .            | 5        |
| <b>6</b> | <b>Resultados da Execução do Programa</b> | <b>5</b> |
| <b>7</b> | <b>Conclusão</b>                          | <b>6</b> |



# 1 Introdução

A programação concorrente permite que múltiplas tarefas sejam executadas simultaneamente, possibilitando melhor aproveitamento dos recursos computacionais. Um dos mecanismos mais utilizados para implementar concorrência é o uso de threads, que representam fluxos independentes de execução dentro de um mesmo programa.

Entretanto, aplicações concorrentes podem apresentar problemas como condições de corrida (race condition) e deadlocks, principalmente quando várias threads tentam acessar recursos compartilhados ao mesmo tempo.

Com o objetivo de compreender esses conceitos na prática, foi desenvolvido um simulador do clássico problema dos Filósofos Jantando utilizando a linguagem Python. O projeto faz uso de threads, semáforos e mutexes para controlar o acesso aos recursos compartilhados representados pelos garfos utilizados pelos filósofos durante a refeição.

O sistema demonstra de forma prática como ocorre a sincronização entre múltiplas threads e como técnicas de controle de concorrência podem evitar conflitos durante a execução paralela.

# 2 Ideias Criativas

Inicialmente, foram consideradas outras possibilidades para trabalhar conceitos de concorrência, como simulações de filas de atendimento e controle de tráfego de veículos.

Entretanto, optou-se pelo problema clássico dos Filósofos Jantando devido à sua importância histórica no estudo de Sistemas Operacionais e sincronização de processos. Além disso, o problema representa de forma clara situações de disputa por recursos compartilhados.

A implementação foi adaptada para tornar a execução mais dinâmica, utilizando filósofos conhecidos da filosofia clássica, como Sócrates, Platão, Aristóteles, Descartes e Nietzsche, permitindo uma visualização mais intuitiva da execução das threads.

Outro ponto importante foi o uso de semáforo para limitar o número de filósofos tentando acessar os garfos simultaneamente, evitando deadlock durante a execução do programa.



### 3 Organização do Projeto

O desenvolvimento do projeto foi dividido entre os integrantes da equipe da seguinte forma:

- Allana: implementação das threads e lógica principal de concorrência;
- Lucas: desenvolvimento da sincronização com semáforos e mutexes;
- José Henrique: testes de execução e análise dos resultados;
- Janiele: elaboração do README e documentação do projeto.

Todos os integrantes participaram da correção de erros, testes finais e ajustes necessários para garantir o correto funcionamento da aplicação.

### 4 Metodologia

O projeto foi desenvolvido em Python utilizando principalmente a biblioteca `threading`, responsável pelo gerenciamento das threads e mecanismos de sincronização.

Também foram utilizadas as bibliotecas:

- `time` — para controle de tempo e simulação das ações dos filósofos;
- `random` — para gerar intervalos aleatórios de pensamento e alimentação.

Cada filósofo foi representado por uma thread independente. Durante a execução, cada thread alterna entre os estados de pensar e comer.

Para que um filósofo possa comer, ele precisa adquirir dois recursos compartilhados: o garfo da esquerda e o garfo da direita. Esses recursos são controlados por mutexes (`threading.Lock`), impedindo que dois filósofos utilizem o mesmo garfo simultaneamente.

Além disso, foi utilizado um semáforo (`threading.Semaphore`) limitando a quantidade de filósofos tentando acessar os recursos ao mesmo tempo, reduzindo o risco de deadlock.

O sistema também registra o tempo total gasto por cada filósofo durante sua execução, permitindo analisar o comportamento das threads.



## 5 Implementação das Funções

### 5.1 Função `run`

A função `run` representa a execução individual de cada filósofo. Ela é executada por uma thread independente.

Inicialmente, o filósofo registra seu tempo de início e entra em um ciclo de pensamento, simulando espera com `time.sleep()`.

Após o período de pensamento, o filósofo solicita acesso aos garfos para comer. O semáforo controla quantos filósofos podem tentar acessar os recursos simultaneamente.

Os garfos são protegidos por mutexes (`Lock`), garantindo acesso exclusivo aos recursos compartilhados.

Após conseguir os dois garfos, o filósofo entra no estado de alimentação, permanece por um tempo aleatório e libera os recursos ao finalizar.

Por fim, o tempo total de execução do filósofo é armazenado no dicionário de resultados.

### 5.2 Função `main`

A função `main` é responsável pelo controle geral do programa.

Nela são criadas:

- A lista de filósofos;
- As threads;
- O dicionário de tempos;
- O controle dos mutexes.

Cada filósofo recebe uma thread própria, iniciada através do método `start()`.

Posteriormente, o método `join()` é utilizado para garantir que o programa principal aguarde a finalização de todas as threads antes de exibir os resultados finais.

Ao término da execução, o sistema imprime o tempo gasto por cada filósofo.



## 5.3 Uso de Semáforos

O semáforo foi utilizado para limitar o número de filósofos que podem tentar acessar os garfos simultaneamente.

No código, o semáforo foi inicializado com:

```
semaforo = threading.Semaphore(NUMERO_DE_FILOSOFOS - 1)
```

Essa abordagem evita deadlock, pois impede que todos os filósofos tentem pegar os garfos ao mesmo tempo.

O semáforo funciona como um mecanismo de controle de acesso concorrente aos recursos compartilhados.

## 5.4 Uso de Mutex (Locks)

Os mutexes representam os garfos compartilhados entre os filósofos.

Cada garfo foi implementado utilizando:

```
threading.Lock()
```

Quando um filósofo deseja comer, ele precisa adquirir os dois locks correspondentes aos garfos vizinhos.

O uso dos mutexes garante exclusão mútua, evitando que dois filósofos utilizem o mesmo garfo simultaneamente.

Isso previne condições de corrida e inconsistências durante a execução concorrente.

# 6 Resultados da Execução do Programa

Durante os testes, cinco filósofos participaram da simulação:

- Sócrates
- Platão
- Aristóteles
- Descartes
- Nietzsche



Cada filósofo executou suas ações de forma concorrente, alternando entre pensar e comer.

A saída do programa apresentou mensagens semelhantes às seguintes:

```
Filósofo Socrates chegou à mesa
Filosofo Socrates esta pensando
Filosofo Platao quer comer
FilosofoAristóteles esta comendo
```

Ao final da execução, o sistema exibiu o tempo total gasto por cada filósofo:

```
Nietzsche demorou 2.542060 segundos
Socrates demorou 3.596913 segundos
Descartes demorou 4.263973 segundos
Aristóteles demorou 6.166511 segundos
Platao demorou 8.002500 segundos
```

Os resultados demonstraram o funcionamento correto da concorrência entre as threads, além da sincronização adequada dos recursos compartilhados.

Também foi possível observar que o uso de semáforos evitou situações de deadlock durante a execução do sistema.

## 7 Conclusão

O desenvolvimento deste projeto possibilitou a aplicação prática de conceitos fundamentais relacionados à programação concorrente e à sincronização de processos, permitindo compreender de forma mais clara o funcionamento de sistemas que executam múltiplas tarefas simultaneamente.

A utilização de threads permitiu a execução paralela das ações dos filósofos, enquanto os mecanismos de sincronização, como mutexes e semáforos, garantiram o controle seguro dos recursos compartilhados, evitando conflitos durante a execução do programa.

Por meio da implementação do problema clássico dos Filósofos Jantando, foi possível analisar na prática conceitos importantes estudados na disciplina de Sistemas Operacionais, tais como concorrência, exclusão mútua, controle de acesso a recursos compartilhados, sincronização de threads, condições de corrida (**race condition**) e prevenção de deadlock.

Além do aprendizado teórico, o projeto proporcionou experiência prática no desenvolvimento de aplicações concorrentes em Python, contribuindo para o aprimoramento da





lógica de programação, da organização do código e da compreensão dos desafios existentes em ambientes paralelos e concorrentes.

Dessa forma, o trabalho reforçou a importância dos mecanismos de sincronização para garantir integridade, segurança e eficiência na execução de sistemas concorrentes.